

PROGRAMMATION ORIENTÉE OBJET

TD 2

1H30

Objectif de la séance

L'objectif de cette séance est d'étudier la classe *string* et les modèles de classe *vector* et *list* de la Standard Template Library (STL). Vous pourrez utiliser les diapositives du cours et vous reporter au site internet <http://www.cplusplus.com/reference/> qui est une documentation générale sur le langage C++.

Au fur et à mesure de la séance, vous devez garder une trace de tous les tests réalisés et des réponses aux questions de l'énoncé en ajoutant les commentaires adaptés dans vos fichiers sources. A la fin de la séance, vous déposerez sur le portail une archive au format zip contenant tous vos fichiers sources (.h et .cpp).

Vous travaillerez avec un unique projet (appelé *td2*) de type « Application console » commun aux 3 exercices. Pour chaque exercice, vous sélectionnerez le fichier qui doit être inclus dans ce projet (les autres seront exclus du projet grâce à l'option "Exclure du projet" accessible par un clic droit sur le fichier à exclure).

Exercice 1 : La classe *string*

La classe *string* de la STL permet de manipuler simplement des chaînes de caractères en gérant l'allocation dynamique du tableau de caractères et en proposant des opérateurs ou des méthodes pour manipuler simplement la chaîne de caractères.

Dans cet exercice, on répondra aux questions en écrivant un programme dans un fichier appelé *test_string.cpp*. Pour répondre à ces questions, il faudra utiliser le plus possible les méthodes ou opérateurs disponibles dans la classe *string*.

a) Ecrire un programme qui déclare un objet de type *string* en l'initialisant avec une constante chaîne, puis affiche le contenu de l'objet et le nombre de caractères de la chaîne.

b) Modifier le programme pour qu'il demande à l'utilisateur de rentrer la chaîne à l'aide de la console (la chaîne pourra comporter des espaces).

c) Compléter le programme pour qu'il compte le nombre de mots se trouvant dans la chaîne et affiche le résultat.

d) Compléter le programme pour qu'il permette à l'utilisateur de rechercher un mot dans la chaîne. Le mot sera demandé à l'utilisateur et, si le mot est présent, le programme devra indiquer la position de ce mot dans la chaîne.

Exercice 2 : Le modèle de classe *vector*

Le modèle de classe *vector* permet de manipuler des tableaux dynamiques d'éléments.

Dans cet exercice, on répondra aux questions en écrivant un programme dans un fichier appelé *test_vector.cpp*. Pour répondre à ces questions, il faudra utiliser le plus possible les méthodes ou opérateurs disponibles dans la classe *vector*.

a) Ecrire un programme qui déclare un vecteur de 5 réels (double), qui remplit ce vecteur avec des valeurs fixées (par exemple 10, 20, 30, 40, 50), et affiche le contenu du tableau.

b) Ecrire une fonction *afficher* qui affiche sur la console le contenu d'un objet *vector<double>*. Modifier le programme précédent pour qu'il utilise cette fonction.

c) Ajouter au programme une boucle *do-while* qui demande à l'utilisateur d'ajouter des valeurs à la fin du vecteur tant que l'utilisateur n'a pas saisi la valeur -1.

Exercice 3 : Le modèle de classe *list*

Le modèle de classe *list* permet de manipuler des listes doublement chaînées d'éléments.

Dans cet exercice, on répondra aux questions en écrivant un programme dans un fichier appelé *test_list.cpp*. Pour répondre à ces questions, il faudra utiliser le plus possible les méthodes ou opérateurs disponibles dans la classe *list*.

a) Ecrire un programme qui déclare une liste d'objets *string* et demande à l'utilisateur des chaînes à insérer au moyen d'une boucle *do-while* (les chaînes ne comporteront pas d'espaces et on devra saisir le mot *fin* pour sortir de la boucle). Ecrire une boucle *for* qui affiche les chaînes contenues dans la liste.

b) Ecrire une fonction *afficher* qui affiche sur la console le contenu d'un objet *list<string>*. Modifier le programme précédent pour qu'il utilise cette fonction (pour pouvoir utiliser un itérateur sur un objet constant, il faut déclarer un itérateur de type *const_iterator*).

c) Ajouter au programme les instructions pour déclarer une nouvelle liste, copier les éléments de la première liste, trier les chaînes par ordre alphabétique et afficher le résultat.

d) Ecrire et tester une fonction *afficherInverse* qui affiche le contenu de la liste en commençant par la fin. Pour cela, il faudra utiliser un itérateur de type *reverse_iterator* (ou *const_reverse_iterator*) et les méthodes *rbegin* et *rend* (cf. documentation du C++).